Ai programs used:

Claude vs. Gemini

(Note: Claude is set for student use purpose where Gemini isn't)

How I accessed them:

Gemini…gemini.google.com on the browser

Claude…claude.ai on windows

Prompt:

Files Attached:
puzzle1.txt
Assignment4 (1).cpp

Prompt used:

Modify the C++ program below. It is a Sudoku solver written by another student.

Existing requirements (all of these must work, even if the current code does not meet them):

- The program must be recursive, object-oriented C++ that uses objects, not just standalone functions.
- It must solve the five Sudoku puzzles in puzzle1.txt through puzzle5.txt. Each file contains a 9x9 grid made up of 3x3 blocks. Filled cells contain digits, and an underscore (_) marks a blank cell.
- Each cell must be filled with a single digit from 1 to 9, and each digit must appear exactly once in every row, every column, and every 3x3 block.
- Use a brute-force depth-first search with backtracking, not logic-based deduction: find an empty cell, work out its candidate values, place the first candidate, and recursively try to solve the puzzle with that cell filled. If no candidates remain or a candidate leads to no solution, back up and try the next candidate. The search ends with a solution when all 81 cells are filled, or determines there is no solution when all candidates fail.
- A puzzle may have more than one solution. The program must find and print all of them, not just the first one.
- The output for each puzzle must include:
  - The puzzle file name (for example, puzzle1.txt)
  - The puzzle from the file, printed out
  - The solution or solutions, or "No solution found" if there are none

New requirements:
1. For each puzzle file, puzzle1.txt through puzzle5.txt, write the solutions to a file named solutionN.txt, where N is the puzzle number.
2. If a puzzle has no solutions, solutionN.txt must contain this single line:
No solution found.

3. If a puzzle has solutions, solutionN.txt must contain all of the solutions, followed by this single line, where n is the number of solutions found:
Solutions found: n
4. Each solution must use the same format as the input puzzle files.

Constraints:
- Modify the existing code. Do not rewrite the program from scratch, and keep the existing structure, function names, and variable names wherever possible.
- Fix any bugs you find in the existing code, and briefly explain each fix.
- Keep everything in a single C++ source file named Assignment5.cpp.
- The code must compile with g++ and run on a Linux server.
- Consider correctness, execution time, space complexity, and maintainability.

Return the complete modified source code.

Generated Code:

Gemini

```
/*
 * Program: EECS 348 Assignment 5 - Sudoku Solver (File Output Update)
 * Description: Solves Sudoku puzzles using recursion and backtracking and
writes solutions to files.
 * Inputs: Sudoku puzzles provided in text files (puzzle1.txt through
puzzle5.txt).
 * Output: Original puzzles, solutions printed to console, and formatted
solution files (solutionN.txt).
 * Author: Dalton Brockert (Modified for Assignment 5)
 * Creation date: October 2026.
 */

#include <iostream> // Provides console input and output.
#include <fstream>  // Allows the program to read and write files.
#include <string>   // Provides the string data type.
#include <vector>   // Provides a resizable list of characters.
#include <cctype>   // Provides isdigit function.

using namespace std; // Avoids writing std:: before standard names.

class SudokuSolver { // Defines an object that solves Sudoku puzzles.
private: // Keeps these members accessible only inside the class.
```

```
    int grid[9][9]; // Stores the 9x9 Sudoku grid.
    string fileName; // Stores the current puzzle filename.
    int solutionCount; // Counts the solutions found.
    ofstream outFile; // Stores the output file stream for solutions.

    bool findEmptyCell(int &row, int &col) { // Finds the next blank cell.
        for (int r = 0; r < 9; r++) { // Checks each row.
            for (int c = 0; c < 9; c++) { // Checks each column.
                if (grid[r][c] == 0) { // Checks whether the cell is
blank.
                    row = r; // Saves the blank cell's row.
                    col = c; // Saves the blank cell's column.
                    return true; // Reports that a blank was found.
                } // Ends the blank-cell check.
            } // Ends the column loop.
        } // Ends the row loop.
        return false; // Reports that no blank cells remain.
    } // Ends findEmptyCell.

    bool isSafe(int row, int col, int digit) { // Checks if a digit can
fit.
        for (int c = 0; c < 9; c++) { // Checks every cell in the row.
            if (grid[row][c] == digit) { // Looks for the same digit.
                return false; // Rejects a duplicate in the row.
            } // Ends the row check.
        } // Ends the row loop.

        for (int r = 0; r < 9; r++) { // Checks every cell in the column.
            if (grid[r][col] == digit) { // Looks for the same digit.
                return false; // Rejects a duplicate in the column.
            } // Ends the column check.
        } // Ends the column loop.

        int blockRow = (row / 3) * 3; // Finds the block's first row.
        int blockCol = (col / 3) * 3; // Finds the block's first column.

        for (int r = blockRow; r < blockRow + 3; r++) { // Checks block
rows.
            for (int c = blockCol; c < blockCol + 3; c++) { // Checks
block columns.
```

```
                if (grid[r][c] == digit) { // Looks for the same digit.
                    return false; // Rejects a duplicate in the block.
                } // Ends the block check.
            } // Ends the inner loop.
        } // Ends the outer loop.

        return true; // Allows the digit if no conflict exists.
    } // Ends isSafe.

    bool initialGridIsValid() { // Checks the original puzzle for
conflicts.
        for (int row = 0; row < 9; row++) { // Visits every row.
            for (int col = 0; col < 9; col++) { // Visits every column.
                int digit = grid[row][col]; // Saves the current cell
value.

                if (digit == 0) { // Checks whether the cell is empty.
                    continue; // Skips empty cells.
                } // Ends the empty-cell check.

                grid[row][col] = 0; // Temporarily clears the current
cell.
                bool valid = isSafe(row, col, digit); // Checks for
conflicts.
                grid[row][col] = digit; // Restores the original digit.

                if (!valid) { // Checks whether a conflict was found.
                    return false; // Rejects the invalid puzzle.
                } // Ends the conflict check.
            } // Ends the column loop.
        } // Ends the row loop.

        return true; // Accepts the puzzle's starting digits.
    } // Ends initialGridIsValid.

    void printGrid() { // Prints the Sudoku grid to console.
        for (int row = 0; row < 9; row++) { // Prints each row.
            if (row % 3 == 0 && row != 0) { // Checks for a block
boundary.
```

```
                cout << "------+-------+------" << endl; // Prints a
divider.
            } // Ends the divider check.

            for (int col = 0; col < 9; col++) { // Prints each cell.
                if (col % 3 == 0 && col != 0) { // Checks for a block
boundary.
                    cout << "| "; // Separates the vertical blocks.
                } // Ends the block-boundary check.

                if (grid[row][col] == 0) { // Checks for an empty cell.
                    cout << "_ "; // Prints an underscore for a blank.
                } else { // Handles cells containing digits.
                    cout << grid[row][col] << " "; // Prints the digit.
                } // Ends the cell-value check.
            } // Ends the column loop.

            cout << endl; // Moves to the next output line.
        } // Ends the row loop.
    } // Ends printGrid.

    void solveRecursive() { // Searches for every possible solution.
        int row; // Stores the next blank cell's row.
        int col; // Stores the next blank cell's column.

        if (!findEmptyCell(row, col)) { // Checks whether the grid is
complete.
            solutionCount++; // Counts the completed solution.
            cout << "\nSolution " << solutionCount << ":" << endl; //
Prints its number.
            printGrid(); // Prints the completed Sudoku grid.

            // Write the solution to the solutionN.txt file in input
format
            for (int r = 0; r < 9; r++) {
                for (int c = 0; c < 9; c++) {
                    if (grid[r][c] == 0) {
                        outFile << "_";
                    } else {
                        outFile << grid[r][c];
```

```
                    }
                    if (c < 8) {
                        outFile << " ";
                    }
                }
                outFile << endl;
            }
            outFile << endl; // Blank line between solutions in output
file

            return; // Returns to search for other solutions.
        } // Ends the completed-grid check.

        for (int digit = 1; digit <= 9; digit++) { // Tries digits 1
through 9.
            if (isSafe(row, col, digit)) { // Checks whether the digit
fits.
                grid[row][col] = digit; // Places the temporary digit.
                solveRecursive(); // Recursively solves the remaining
cells.
                grid[row][col] = 0; // Removes the digit to backtrack.
            } // Ends the valid-digit check.
        } // Ends the digit loop.
    } // Ends solveRecursive.

public: // Makes these members accessible outside the class.
    SudokuSolver() { // Creates and initializes a solver object.
        solutionCount = 0; // Starts the solution count at zero.

        for (int row = 0; row < 9; row++) { // Visits every row.
            for (int col = 0; col < 9; col++) { // Visits every column.
                grid[row][col] = 0; // Marks every cell as empty.
            } // Ends the column loop.
        } // Ends the row loop.
    } // Ends the constructor.

    bool loadFromFile(const string &name) { // Reads a puzzle from a file.
        fileName = name; // Saves the filename.
        ifstream inputFile(fileName); // Opens the puzzle file.
```

```
        if (!inputFile.is_open()) { // Checks whether the file opened.
            cerr << "Error: Could not open " << fileName << endl; //
Prints an error.
            return false; // Reports that loading failed.
        } // Ends the file-opening check.

        vector<char> cells; // Stores the puzzle's cell characters.
        char character; // Stores each character read from the file.

        while (inputFile.get(character)) { // Reads one character at a
time.
            if ((character >= '1' && character <= '9') || character ==
'_') { // Checks for a cell.
                cells.push_back(character); // Saves the cell character.
            } else if (character != ' ' && character != '\t' && character
!= '\n' && character != '\r') { // Checks for invalid characters.
                cerr << "Error: Invalid character in " << fileName <<
endl; // Prints an error.
                inputFile.close(); // Closes the input file.
                return false; // Reports that loading failed.
            } // Ends the character check.
        } // Ends the file-reading loop.

        inputFile.close(); // Closes the file after reading.

        if (cells.size() != 81) { // Checks for exactly 81 cells.
            cerr << "Error: " << fileName << " must contain exactly 81
cells; found " << cells.size() << "." << endl; // Explains the size error.
            return false; // Rejects the incorrectly sized puzzle.
        } // Ends the cell-count check.

        int index = 0; // Tracks the next character to copy.
        for (int row = 0; row < 9; row++) { // Fills each row.
            for (int col = 0; col < 9; col++) { // Fills each column.
                if (cells[index] == '_') { // Checks for a blank
character.
                    grid[row][col] = 0; // Stores blanks as zero.
                } else { // Handles digit characters.
                    grid[row][col] = cells[index] - '0'; // Converts the
character to an integer.
```

```
                } // Ends the cell conversion.

                index++; // Moves to the next character.
            } // Ends the column loop.
        } // Ends the row loop.

        return true; // Reports that loading succeeded.
    } // Ends loadFromFile.

    void solve() { // Prints, solves the current puzzle, and writes
outputs.
        cout << "\n========================================" << endl; //
Prints a top divider.
        cout << "Puzzle file: " << fileName << endl; // Prints the
filename.
        cout << "========================================" << endl; //
Prints a bottom divider.

        cout << "\nOriginal puzzle:" << endl; // Labels the original grid.
        printGrid(); // Displays the unsolved puzzle.

        // Determine solution filename (solutionN.txt) from input filename
(puzzleN.txt)
        string solFileName = "solution1.txt";
        size_t numPos = fileName.find_first_of("0123456789");
        if (numPos != string::npos) {
            string numStr = "";
            while (numPos < fileName.length() &&
isdigit(fileName[numPos])) {
                numStr += fileName[numPos];
                numPos++;
            }
            solFileName = "solution" + numStr + ".txt";
        }

        outFile.open(solFileName); // Opens output file.
        if (!outFile.is_open()) {
            cerr << "Error: Could not open output file " << solFileName <<
endl;
            return;
```

```
        }

        if (!initialGridIsValid()) { // Checks for conflicting starting
digits.
            cout << "\nPuzzle has conflicting starting digits." << endl;
// Reports an invalid puzzle.
            outFile << "No solution found." << endl;
            outFile.close();
            return; // Stops solving this puzzle.
        } // Ends the starting-grid check.

        solutionCount = 0; // Resets the solution count.
        solveRecursive(); // Searches for every solution.

        if (solutionCount == 0) { // Checks whether any solution was
found.
            cout << "\nNo solution found" << endl; // Reports that no
solution exists.
            outFile << "No solution found." << endl;
        } else { // Handles puzzles with solutions.
            cout << "\nTotal solutions found: " << solutionCount << endl;
// Prints the total.
            outFile << "Solutions found: " << solutionCount << endl;
        } // Ends the solution-count check.

        outFile.close(); // Closes the solution file.
    } // Ends solve.
}; // Ends the SudokuSolver class.

int main() { // Starts the program.
    string puzzleFiles[] = { // Lists the five input files.
        "puzzle1.txt", // First Sudoku puzzle.
        "puzzle2.txt", // Second Sudoku puzzle.
        "puzzle3.txt", // Third Sudoku puzzle.
        "puzzle4.txt", // Fourth Sudoku puzzle.
        "puzzle5.txt"  // Fifth Sudoku puzzle.
    }; // Ends the filename list.

    for (int i = 0; i < 5; i++) { // Processes all five puzzles.
        SudokuSolver solver; // Creates a solver for this puzzle.
```

```
        if (solver.loadFromFile(puzzleFiles[i])) { // Loads the current
puzzle.
            solver.solve(); // Solves it and writes to file if loading
succeeded.
        } // Ends the loading check.
    } // Ends the puzzle loop.

    return 0; // Ends the program successfully.
} // Ends main.
```

Claude

```
/*
 * Program: EECS 348 Assignment 5 - Sudoku Solver
 * Description: Solves Sudoku puzzles using recursion and backtracking,
and writes each puzzle's solutions to a file.
 * Inputs: puzzle1.txt through puzzle5.txt, each with 81 cells (digits 1-9
and underscores for blanks).
 * Output: Original puzzles, all solutions, or "No solution found" on the
console, plus solution1.txt through solution5.txt.
 * Collaborators: None.
 * Other sources: None.
 * Author: Dalton Brockert
 * Modified by: Jun Brewer
 * Creation date: October 15, 2026.
 * Modification date: October 29, 2026.
 * Code origin: Generated by ChatGPT, then reviewed and improved for
Assignment 5 with Claude.
 */

#include <iostream> // Provides console input and output.
#include <fstream> // Allows the program to read and write files.
#include <string> // Provides the string data type.
#include <vector> // Provides a resizable list of characters.

using namespace std; // Avoids writing std:: before standard names.

class SudokuSolver { // Defines an object that solves Sudoku puzzles.
```

```
private: // Keeps these members accessible only inside the class.
    int grid[9][9]; // Stores the 9x9 Sudoku grid.
    string fileName; // Stores the current puzzle filename.
    int solutionCount; // Counts the solutions found.
    ofstream solutionFile; // Writes solutions as they are found, so none
are kept in memory.

    bool findEmptyCell(int &row, int &col) { // Finds the next blank cell.
        for (int r = 0; r < 9; r++) { // Checks each row.
            for (int c = 0; c < 9; c++) { // Checks each column.
                if (grid[r][c] == 0) { // Checks whether the cell is
blank.
                    row = r; // Saves the blank cell's row.
                    col = c; // Saves the blank cell's column.
                    return true; // Reports that a blank was found.
                } // Ends the blank-cell check.
            } // Ends the column loop.
        } // Ends the row loop.
        return false; // Reports that no blank cells remain.
    } // Ends findEmptyCell.

    bool isSafe(int row, int col, int digit) { // Checks if a digit can
fit.
        for (int c = 0; c < 9; c++) { // Checks every cell in the row.
            if (grid[row][c] == digit) { // Looks for the same digit.
                return false; // Rejects a duplicate in the row.
            } // Ends the row check.
        } // Ends the row loop.

        for (int r = 0; r < 9; r++) { // Checks every cell in the column.
            if (grid[r][col] == digit) { // Looks for the same digit.
                return false; // Rejects a duplicate in the column.
            } // Ends the column check.
        } // Ends the column loop.

        int blockRow = (row / 3) * 3; // Finds the block's first row.
        int blockCol = (col / 3) * 3; // Finds the block's first column.

        for (int r = blockRow; r < blockRow + 3; r++) { // Checks block
rows.
```

```
            for (int c = blockCol; c < blockCol + 3; c++) { // Checks
block columns.
                if (grid[r][c] == digit) { // Looks for the same digit.
                    return false; // Rejects a duplicate in the block.
                } // Ends the block check.
            } // Ends the inner loop.
        } // Ends the outer loop.

        return true; // Allows the digit if no conflict exists.
    } // Ends isSafe.

    bool initialGridIsValid() { // Checks the original puzzle for
conflicts.
        for (int row = 0; row < 9; row++) { // Visits every row.
            for (int col = 0; col < 9; col++) { // Visits every column.
                int digit = grid[row][col]; // Saves the current cell
value.

                if (digit == 0) { // Checks whether the cell is empty.
                    continue; // Skips empty cells.
                } // Ends the empty-cell check.

                grid[row][col] = 0; // Temporarily clears the current
cell.
                bool valid = isSafe(row, col, digit); // Checks for
conflicts.
                grid[row][col] = digit; // Restores the original digit.

                if (!valid) { // Checks whether a conflict was found.
                    return false; // Rejects the invalid puzzle.
                } // Ends the conflict check.
            } // Ends the column loop.
        } // Ends the row loop.

        return true; // Accepts the puzzle's starting digits.
    } // Ends initialGridIsValid.

    void printGrid(ostream &out) { // Prints the grid in the puzzle file
format.
        for (int row = 0; row < 9; row++) { // Prints each row.
```

```
            for (int col = 0; col < 9; col++) { // Prints each cell.
                if (grid[row][col] == 0) { // Checks for an empty cell.
                    out << "_ "; // Prints an underscore for a blank.
                } else { // Handles cells containing digits.
                    out << grid[row][col] << " "; // Prints the digit.
                } // Ends the cell-value check.
            } // Ends the column loop.

            out << '\n'; // Moves to the next line without a slow flush.
        } // Ends the row loop.
    } // Ends printGrid.

    void solveRecursive() { // Searches for every possible solution.
        int row; // Stores the next blank cell's row.
        int col; // Stores the next blank cell's column.

        if (!findEmptyCell(row, col)) { // Checks whether the grid is
complete.
            solutionCount++; // Counts the completed solution.
            cout << "\nSolution " << solutionCount << ":\n"; // Prints its
number.
            printGrid(cout); // Prints the completed Sudoku grid.

            if (solutionCount > 1) { // Checks for an earlier solution in
the file.
                solutionFile << '\n'; // Separates it from the previous
solution.
            } // Ends the separator check.

            printGrid(solutionFile); // Writes the completed Sudoku grid.
            return; // Returns to search for other solutions.
        } // Ends the completed-grid check.

        for (int digit = 1; digit <= 9; digit++) { // Tries digits 1
through 9.
            if (isSafe(row, col, digit)) { // Checks whether the digit
fits.
                grid[row][col] = digit; // Places the temporary digit.
                solveRecursive(); // Recursively solves the remaining
cells.
```

```
                grid[row][col] = 0; // Removes the digit to backtrack.
            } // Ends the valid-digit check.
        } // Ends the digit loop.
    } // Ends solveRecursive.

public: // Makes these members accessible outside the class.
    SudokuSolver() { // Creates and initializes a solver object.
        solutionCount = 0; // Starts the solution count at zero.

        for (int row = 0; row < 9; row++) { // Visits every row.
            for (int col = 0; col < 9; col++) { // Visits every column.
                grid[row][col] = 0; // Marks every cell as empty.
            } // Ends the column loop.
        } // Ends the row loop.
    } // Ends the constructor.

    bool loadFromFile(const string &name) { // Reads a puzzle from a file.
        fileName = name; // Saves the filename.
        ifstream inputFile(fileName); // Opens the puzzle file.

        if (!inputFile.is_open()) { // Checks whether the file opened.
            cerr << "Error: Could not open " << fileName << endl; //
Prints an error.
            return false; // Reports that loading failed.
        } // Ends the file-opening check.

        vector<char> cells; // Stores the puzzle's cell characters.
        char character; // Stores each character read from the file.

        while (inputFile.get(character)) { // Reads one character at a
time.
            if ((character >= '1' && character <= '9') || character ==
'_') { // Checks for a cell.
                cells.push_back(character); // Saves the cell character.
            } else if (character != ' ' && character != '\t' && character
!= '\n' && character != '\r') { // Checks for invalid characters.
                cerr << "Error: Invalid character in " << fileName <<
endl; // Prints an error.
                inputFile.close(); // Closes the input file.
                return false; // Reports that loading failed.
```

```
            } // Ends the character check.
        } // Ends the file-reading loop.

        inputFile.close(); // Closes the file after reading.

        if (cells.size() != 81) { // Checks for exactly 81 cells.
            cerr << "Error: " << fileName << " must contain exactly 81
cells; found " << cells.size() << "." << endl; // Explains the size error.
            return false; // Rejects the incorrectly sized puzzle.
        } // Ends the cell-count check.

        int index = 0; // Tracks the next character to copy.
        for (int row = 0; row < 9; row++) { // Fills each row.
            for (int col = 0; col < 9; col++) { // Fills each column.
                if (cells[index] == '_') { // Checks for a blank
character.
                    grid[row][col] = 0; // Stores blanks as zero.
                } else { // Handles digit characters.
                    grid[row][col] = cells[index] - '0'; // Converts the
character to an integer.
                } // Ends the cell conversion.

                index++; // Moves to the next character.
            } // Ends the column loop.
        } // Ends the row loop.

        return true; // Reports that loading succeeded.
    } // Ends loadFromFile.

    bool solve(const string &outputName) { // Prints, solves, and saves
the current puzzle.
        cout << "\n========================================\n"; // Prints
a top divider.
        cout << "Puzzle file: " << fileName << '\n'; // Prints the
filename.
        cout << "========================================\n"; // Prints a
bottom divider.

        cout << "\nOriginal puzzle:\n"; // Labels the original grid.
        printGrid(cout); // Displays the unsolved puzzle.
```

```
        solutionFile.open(outputName); // Creates the solution file.

        if (!solutionFile.is_open()) { // Checks whether the file opened.
            cerr << "Error: Could not create " << outputName << endl; //
Prints an error.
            return false; // Reports that solving failed.
        } // Ends the file-opening check.

        solutionCount = 0; // Resets the solution count.

        if (initialGridIsValid()) { // Checks for conflicting starting
digits.
            solveRecursive(); // Searches for every solution.
        } else { // Handles a puzzle that cannot be solved.
            cout << "\nPuzzle has conflicting starting digits.\n"; //
Reports an invalid puzzle.
        } // Ends the starting-grid check.

        if (solutionCount == 0) { // Checks whether any solution was
found.
            cout << "\nNo solution found" << endl; // Reports that no
solution exists.
            solutionFile << "No solution found.\n"; // Records that no
solution exists.
        } else { // Handles puzzles with solutions.
            cout << "\nTotal solutions found: " << solutionCount << endl;
// Prints the total.
            solutionFile << "Solutions found: " << solutionCount << '\n';
// Records the total.
        } // Ends the solution-count check.

        solutionFile.close(); // Saves everything to the solution file.

        if (solutionFile.fail()) { // Checks whether any write failed.
            cerr << "Error: Could not write " << outputName << endl; //
Prints an error.
            return false; // Reports that solving failed.
        } // Ends the write check.
```

```
        return true; // Reports that the solutions were saved.
    } // Ends solve.
}; // Ends the SudokuSolver class.

int main() { // Starts the program.
    string puzzleFiles[] = { // Lists the five input files.
        "puzzle1.txt", // First Sudoku puzzle.
        "puzzle2.txt", // Second Sudoku puzzle.
        "puzzle3.txt", // Third Sudoku puzzle.
        "puzzle4.txt", // Fourth Sudoku puzzle.
        "puzzle5.txt" // Fifth Sudoku puzzle.
    }; // Ends the filename list.

    int exitCode = 0; // Stays zero only if every puzzle is processed.

    for (int i = 0; i < 5; i++) { // Processes all five puzzles.
        SudokuSolver solver; // Creates a solver for this puzzle.
        string solutionName = "solution" + to_string(i + 1) + ".txt"; //
Builds the output filename.

        if (!solver.loadFromFile(puzzleFiles[i]) ||
!solver.solve(solutionName)) { // Loads, then solves the puzzle.
            exitCode = 1; // Records the failure and moves on to the next
puzzle.
        } // Ends the failure check.
    } // Ends the puzzle loop.

    return exitCode; // Ends the program, reporting any failure.
} // Ends main.
```

Analysis:

1. Correctness
   Both programs keep the same recursive backtracking solver, so they get the same results on all five puzzles: one solution each for puzzles 1 through 3, no solution for puzzle 4, and two solutions for puzzle 5. Both also create solution1.txt through solution5.txt, and the digits in those files match exactly. The only difference in the files is whitespace. Claude keeps the space at the end of each line, which matches the puzzle files, while Gemini removes it and adds a blank line before the “Solutions found” line.

Claude's version is more robust when something goes wrong. It checks for failed writes and returns an exit code of 1 if any puzzle fails. Gemini's version only checks that the file opened, and it returns 0 even when a puzzle file is missing. Claude's version did lose something though: it uses one printGrid() for the console and the file, so the 3x3 dividers are gone from the console output. Gemini's console output is identical to the original program. Neither version clears an old solution file when its puzzle fails to load, so old results can be left behind.

2. Execution Time
   The search is the same in both programs, and both finish the five puzzles in about 0.15 seconds. The difference only shows up in the output. Gemini uses endl, which flushes the stream on every line, while Claude uses '\n'. On a test puzzle with 526,176 solutions and the console output sent to a file, Claude took 11.3 seconds and Gemini took 40.0 seconds. Part of that gap comes from Claude printing about 30% less to the console now that the dividers are gone.

3. Space Complexity
   Identical for both programs. Both solve in place on the 9x9 grid and write each solution as it is found instead of storing it, so memory does not grow with the number of solutions.

4. Maintainability
   Claude reuses printGrid(ostream &out) for both the console and the file, so the grid format is written in one place. Gemini adds a second printing loop inside solveRecursive() that repeats that work. Claude builds the output filename in main from the loop index, which is simple. Gemini parses the number out of the puzzle filename, which takes more code and an extra include, and it would pick the wrong number for a path like set2/puzzle1.txt. Claude also follows the existing style of commenting every line, while most of Gemini's new lines have no comment. Claude keeps the full header and adds who modified the program and when. Gemini's header drops the Collaborators, Other sources, and Code origin lines. One problem with Claude's header is that it says "Other sources: None" while the Code origin line names Claude.

Code base: Claude's version

It has better error handling, less repeated code, and faster output.

Edits:

1. Correctness
   Fixed the places where Claude's version handled problems badly. A solution file now gets deleted when its puzzle fails to load, so old results aren't left behind. A puzzle with conflicting digits now gets one console message instead of two, and "No solution found." has a period on both the console and in the file. The program also no longer stops early when a solution file can't be created. It shows the answer on the console and still returns an exit code of 1. This early return was taken out of solve().

```
    if (!solutionFile.is_open()) {
        cerr << "Error: Could not create " << outputName << endl;
        return false; // stopped before solving
    }
```

2. Execution Time
   Changed isSafe() to look up three tables of used digits (rowUsed, colUsed, and blockUsed) instead of scanning the row, column, and block on every check. Also passed the current cell into solveRecursive(), so findEmptyCell() no longer starts over from the top of the grid. On a hard test puzzle with 17 starting digits, the time went from about 6.5 seconds to about 1.7 seconds. On the test puzzle with 526,176 solutions, it went from about 11 seconds to about 9 seconds, and the solutions stayed in the same order. Filling in the most constrained cell first was tested but not used. It solved the hard puzzle in about 0.3 seconds, but it was slower on the 526,176-solution puzzle and it changed the order of the solutions.

3. Space Complexity
   Removed the vector<char> from loadFromFile(). Cells now go straight into the grid as they are read, so memory no longer grows with the size of the input file. The used-digit tables add a small amount of memory, but it is the same for every puzzle.

4. Maintainability
   Added the named constants GRID_SIZE, BLOCK_SIZE, and CELL_COUNT in place of 9, 3, and 81, and changed main() to loop over the puzzle list instead of hardcoding 5. Moved work out of solve() and solveRecursive() into printHeader(), reportSolution(), and reportResult(). Marked the functions that only read data as const, and removed <vector> and the inputFile.close() calls that weren't needed. initialGridIsValid() also no longer has to blank out a cell and put it back. Fixed the header comment so that "Other sources" lists the AI tools used instead of saying None.